Architectural Blueprint for a High-Performance, Multimodal WhatsApp Recruitment AI Agent

The deployment of conversational artificial intelligence within the recruitment lifecycle presents unique engineering challenges, particularly when interfacing with diverse user demographics, complex linguistic topographies, and legacy Applicant Tracking Systems (ATS). The existing WhatsApp chatbot architecture demonstrates significant structural deficiencies, primarily characterized by unacceptable latency times extending beyond a minute, poor comprehension of low-resource and code-mixed languages such as Sinhala, Tamil, Singlish, and Tanglish, rigid and static knowledge retrieval mechanisms, and inadequate multimodal data ingestion capabilities for handling applicant resumes and voice notes. To elevate this system to an enterprise-grade standard capable of functioning as an autonomous recruitment agent, a comprehensive architectural redesign is required. This extensive report provides a meticulous, expert-level blueprint for reconstructing the WhatsApp recruitment chatbot. The proposed architecture addresses five core pillars: advanced multilingual code-mixed language comprehension, high-fidelity multimodal processing covering voice and optical character recognition, asynchronous event-driven microservices to eradicate latency, dynamic Retrieval-Augmented Generation (RAG) for real-time job synchronization, and seamless ATS integration via structured data orchestration.

Advanced Multilingual and Code-Mixed Language Comprehension

The linguistic landscape of the Sri Lankan and interconnected South Asian demographics involves complex morphological structures that routinely break traditional conversational AI systems. Sinhala, an Indo-Aryan language, and Tamil, a Dravidian language, are highly agglutinative, meaning words are formed by stringing together multiple morphemes to convey complex grammatical concepts.

This agglutinative nature complicates standard tokenization pipelines, as a single word can represent an entire sentence in English, leading to an explosion of out-of-vocabulary tokens in models not specifically trained on these languages. Furthermore, digital communication across platforms like WhatsApp predominantly features code-mixing and code-switching, which is the fluid alternation between English and native languages within a single utterance, often transliterated entirely into Latin script to form Singlish and Tanglish. Traditional Natural Language Processing (NLP) models, which are primarily trained on monolingual English corpora, experience severe performance degradation when processing these syntactic mismatches, orthographic variability, and phonetic ambiguities.

The Challenge of Code-Mixed Topographies

Code-mixing introduces severe challenges for standard Large Language Models (LLMs) because it bypasses conventional grammatical rules and localized vocabulary dictionaries. A user might input a phrase blending English terminology with Sinhala grammar, completely in Latin script, which a standard English model will misinterpret as typographical errors, while a standard Sinhala Unicode model will fail to parse the Latin characters entirely. The integration of specific subword tokenization techniques, such as Byte-Pair Encoding (BPE) or WordPiece, is critical for mitigating this. These mechanisms deconstruct unrecognized code-mixed terminology into manageable subwords, allowing the model to process out-of-vocabulary variations dynamically and infer meaning from contextual embeddings.

For extensive NLP tasks in this domain, leveraging localized Python libraries provides a necessary preprocessing layer before the text reaches the primary LLM. Libraries such as SinLingua for Sinhala and ThamizhiLIP for Tamil enable hybrid rule-based and machine-learning text preprocessing, transliteration, and morphological analysis. Transliteration pipelines can convert Romanized Singlish and Tanglish inputs back into native Unicode scripts, which significantly stabilizes the input for downstream sentiment analysis, intent classification, and entity extraction. Additional resources like the Indic NLP Library facilitate text normalization, removing non-spacing characters and handling typing inconsistencies, ensuring that the data fed into the generation model is clean and standardized.

Optimal Large Language Model Selection

The selection of the foundational LLM dictates the system's ability to navigate conversational nuance, extract relevant candidate entities, and maintain a natural, human-like persona. For this sophisticated architecture, a dual-model routing strategy proves optimal, balancing open-weight localized models for basic interactions with state-of-the-art commercial APIs for complex reasoning tasks.

Recent advancements in localized open-weight LLMs have produced models continually pre-trained on South Asian corpora, explicitly designed to bridge the resource gap. SinLlama, developed by extending the Llama-3-8B architecture, represents a breakthrough in Sinhala NLP. The developers augmented the LLM tokenizer with Sinhala-specific vocabulary and performed continual pre-training on a ten-million sentence Sinhala corpus. Empirical evidence demonstrates that SinLlama outperforms both the base and instruct variants of Llama-3-8B by a significant margin on text classification benchmarks, making it the premier decoder-based open-source model for explicit Sinhala support. For Tamil comprehension, models like Tamil-Llama provide a LoRA-adapted framework with an expanded vocabulary. Additionally, the Qwen3 series, particularly the Qwen3-235B and Qwen3-8B models, utilize a Mixture-of-Experts (MoE) architecture that demonstrates exceptional multilingual instruction following, explicitly supporting Tamil and seamlessly handling code-switched dialogues and multi-turn contextual conversations.

While open-weight models offer data sovereignty and lower long-term inference costs, commercial proprietary models provide unmatched reasoning capabilities necessary for complex ATS orchestration and JSON generation. A comparative analysis of leading LLMs for this specialized recruitment context reveals distinct operational advantages.

Language Model	Architectural Strengths	Multilingual & Code-Mixed Performance	Ideal Recruitment Use Case
Gemini 2.5 Pro	Massive context window (up to 1M+ tokens), multimodal natively processing audio and text simultaneously.	Top performer on Unicode-native South Asian scripts; excels in multilingual reasoning and high-fidelity code parsing.	Cross-referencing lengthy CVs against extensive job descriptions; complex JSON data extraction.

GPT-4o	Highly optimized for low-latency output and precise instruction following; natively multimodal.	Strong fluency across major languages; highly reliable in maintaining structured output schemas.	Agentic decision making, intent routing, and real-time conversational management.
SinLlama (8B)	Decoder-based architecture specifically continually pre-trained on 10 million Sinhala sentences.	Exceptional performance on Sinhala classification tasks; overcomes typical Llama-3 Low-Resource Language deficits.	Localized, privacy-first processing of Sinhala-dominant candidate screening conversations.
Qwen3 (235B / 8B)	Mixture-of-Experts (MoE) design allowing efficient scaling; specifically optimized for Asian dialects.	Flawlessly handles Tamil dialogue generation and complex multilingual reasoning tasks.	Handling Tanglish queries and providing dynamic, culturally nuanced onboarding support.
DeepSeek-V3	Highly efficient MoE architecture maximizing throughput while lowering computational overhead.	Superior capabilities in technical translation and preserving logical relationships during language switching.	Translating highly technical IT or engineering job descriptions into localized languages accurately.

A highly effective routing architecture involves utilizing a faster, localized model like Qwen3-8B or SinLlama for basic conversational chitchat, initial language detection, and frequently asked questions, while routing complex tasks such as CV extraction, dynamic knowledge base retrieval, and final JSON schema generation to Gemini 2.5 Pro or GPT-4o. This hybrid approach optimizes both cost and response quality.

Eradicating Latency: Asynchronous Architecture and Inference Optimization

A response latency exceeding one minute is catastrophic for user experience, completely shattering the illusion of a conversational agent and leading to severe user frustration and application abandonment. In modern generative AI systems, latency is bifurcated into two primary components: architectural blocking caused

by synchronous network bottlenecks, and the inherent inference generation times of the LLMs themselves. Both layers require rigorous re-engineering to achieve sub-five-second responses.

Decoupling the Backend with FastAPI, Celery, and Redis

The current implementation of the chatbot likely relies on a synchronous request-response cycle, where the WhatsApp incoming webhook forces the web server to wait for the entire sequential process—LLM text generation, optical character recognition on uploaded images, vector database querying, and ATS API integration—to conclude before returning an HTTP response to Meta's servers. This synchronous trap exhausts worker threads rapidly; if multiple candidates apply simultaneously, the server crashes or times out.

To achieve high-throughput concurrency, the backend must be refactored into a fully asynchronous, event-driven microservices architecture utilizing FastAPI, Redis, and Celery. FastAPI acts as the external gateway, providing native asynchronous capabilities via Python's `async def` and `await` syntax, allowing the ASGI server to handle thousands of concurrent WhatsApp webhooks without blocking the primary event loop. When a candidate's message arrives, FastAPI immediately acknowledges the receipt to the WhatsApp API, validating the payload via strict Pydantic models to prevent malformed data from entering the system, and instantly pushes the processing instructions to a message broker.

Redis is deployed as an in-memory message broker and data store, managing the incoming tasks in highly optimized queues. It serves a critical dual purpose within this architecture: acting as the transport layer to route tasks to available computational nodes, and acting as the result backend to store the final generated LLM responses or extracted candidate data temporarily.

Celery worker processes operate completely independently of the web server's request cycle. These workers continuously poll the Redis queues, pulling tasks and executing the heavy computational workloads in the background. A Celery worker will manage the interaction with the LLM, run OCR algorithms on a submitted CV, or query the RAG vector database, writing the final structured result back to Redis. Once the task is complete, a secondary lightweight dispatcher service or asynchronous webhook pushes the finalized text message or document link back to the user via the WhatsApp Cloud API. This decoupled architecture

ensures infinite horizontal scalability. If the recruitment drive experiences a sudden surge in applications, systems administrators can programmatically spin up additional Celery worker containers to absorb the computational load without the FastAPI gateway ever failing to acknowledge an incoming message.

LLM Inference Latency Optimization

Beyond rectifying architectural bottlenecks, the inference process of the generative model itself must be hyper-optimized. User-perceived speed is governed primarily by Time-To-First-Token (TTFT), the delay before the very first word appears, and Time-To-Last-Token (TTLT), the total time required to generate the complete response. Optimizing these metrics requires a multifaceted approach involving prompt engineering, hardware optimization, and advanced execution strategies.

Optimization Strategy	Operational Mechanism	Impact on System Latency
Output Token Reduction	Output tokens demand significantly higher computational time than input tokens (often 200x more processing time). Instructing the model to "be concise," applying explicit brevity constraints, or limiting JSON schema keys (e.g., mapping `candidate_first_name` to `fn`) drastically reduces the volume of tokens generated.	Cutting output tokens by 50% reliably yields a proportional 40-50% reduction in generation latency.
Semantic Caching	Implementing an intelligent caching layer, utilizing Redis or specialized API gateways, to store vector embeddings of frequently asked questions (e.g., "Where is your office?"). If a new query semantically matches a cached intent within a defined threshold, the pre-generated response is served instantaneously without invoking the LLM.	Reduces TTFT to near-zero for high-frequency queries and cuts API inference costs by up to 30%.
Parallel Execution & Prompt Consolidation	Eliminating sequential API calls by combining multiple operational steps into a single comprehensive prompt. Instead of querying the LLM to extract skills, waiting for the	Eliminates unnecessary network chatter and connection overhead, saving 0.5–1.5 seconds

	response, and querying again to match jobs, a single prompt requests all operations concurrently, outputting results in a unified, enumerated JSON structure.	per eliminated round-trip.
Response Streaming	Configuring the LLM API to yield partial text outputs to the user as they are generated, rather than buffering the entire completion. While the overall TTLT remains unaffected, the perceived TTFT drops to milliseconds.	Transforms the user experience from waiting for a stagnant loading state to observing real-time typing, eliminating perceived lag.
Asynchronous API Invocation	Utilizing asynchronous Python libraries (e.g., `asyncio.gather`) to issue multiple LLM requests concurrently. When evaluating a candidate against three different job descriptions, the system fires all three evaluations simultaneously rather than waiting for each to complete sequentially.	Maximizes system throughput and reduces total processing time by a factor of the number of parallel requests executed.

Applying these principles holistically transforms the chatbot from a sluggish, processing-heavy application into a highly responsive, real-time conversational agent capable of maintaining fluid dialog with candidates.

Multimodal Data Ingestion: Voice and Optical Character Recognition

A modern digital recruitment agent must transcend text-based interactions. Candidates operating via mobile platforms frequently communicate utilizing voice notes for convenience and submit critical onboarding documentation—resumes, identification cards, and academic certificates—as complex PDFs or hastily captured smartphone photographs. The system requires highly specialized, robust multimodal ingestion pipelines capable of extracting flawless data from these diverse inputs regardless of noise or formatting.

High-Accuracy Speech-to-Text (STT) Optimization

Processing Sinhala and Tamil audio transmitted via WhatsApp voice notes introduces severe acoustic and linguistic challenges. These challenges include environmental background noise, varying hardware degradation across different

mobile microphones, and the heavy use of intra-sentential code-mixing where a candidate might switch between Tamil and English within the same breath. Standard commercial Speech-to-Text models often struggle profoundly with Low-Resource Languages, exhibiting unacceptably high Word Error Rates (WER) that render the transcriptions useless for automated data extraction.

The open-source landscape is dominated by OpenAI's Whisper model, which provides robust baseline multilingual capabilities. However, deploying zero-shot Whisper on South Asian languages reveals significant flaws. Research indicates a critical bug in OpenAI's text normalization process for Brahmic scripts, which artificially inflates reported accuracy; in practice, zero-shot Whisper often hallucinates, repeats phrases, or misinterprets phonetically complex utterances. To leverage Whisper effectively, the architecture must implement fine-tuned variants. Models such as `whisper-small-sinhala`, which have been explicitly fine-tuned on localized datasets like the Mozilla Common Voice 11.0 corpus, drastically reduce the WER for Sinhala speech. To deploy this cost-effectively and maintain low latency, these models must be quantized and executed using highly optimized inference engines like CTranslate2 or WhisperX, which additionally provides speaker diarization capabilities to distinguish between multiple voices if a candidate uploads a recorded interview segment.

For enterprise-grade reliability and ease of integration, commercial APIs specifically architected around Indian Subcontinent linguistics offer superior out-of-the-box performance, often eliminating the need for complex local GPU hosting.

Speech-to-Text Engine	**Key Differentiators**	**Code-Mixing & Dialect Capabilities**	**Suitability for Recruitment Architecture**
Sarvam AI	Purpose-built for Indic languages; highly optimized for telephony audio (8kHz) and extreme background noise typical of mobile calls.	Flawless, seamless code-mixing support; understands mid-sentence switches between Hindi, English, and regional languages.	Excellent for processing noisy WhatsApp voice notes; preserves precise formatting and number normalization for extracting contact details.

Soniox	Claims native-speaker accuracy in real-world environments; features advanced conversational endpointing to detect natural speech flow.	Exceptional handling of Tanglish and Hinglish; specifically praised for accurate alphanumeric detection (spelling out emails or IDs).	Ideal for real-time live voice agent interactions and capturing complex candidate identifiers spoken in mixed languages.
Google Chirp 3	Massive multilingual coverage (100+ languages); deep integration with the Google Cloud ecosystem; robust speaker diarization.	Strong performance across variants but can suffer from latency issues in streaming contexts and higher operational costs.	Suitable for batch processing of large candidate video submissions or long-form recorded interviews where real-time speed is secondary.
ElevenLabs Scribe	Delivers incredibly low WER on benchmarks (e.g., 5.9% on FLEURS for Tamil); features character-level timestamps and audio-event tagging.	High precision in capturing nuances; outperforms standard Whisper models significantly in independent benchmarking.	Best utilized when exact transcript fidelity is paramount, such as documenting formal candidate statements or legal compliance recordings.

Optical Character Recognition (OCR) for Complex Scripts

When candidates upload resumes or identification documents as images, the system must extract the data flawlessly to populate the ATS. Sinhala and Tamil scripts are visually and structurally complex, featuring curved geometries, interconnected modifiers, and intricate vowel diacritics. Traditional OCR engines like Tesseract, while ubiquitous, are largely ineffective in zero-shot scenarios for these languages, often resulting in high Character Error Rates (CER) and requiring extensive, brittle pre-processing (binarization, edge detection) and post-processing steps that still yield suboptimal results.

Recent academic benchmarking of modern OCR architectures on Low-Resource Languages provides a definitive path forward for this system. For extracting data from Sinhala documents, Surya OCR has emerged as the undisputed state-of-the-art solution. Comprehensive evaluations demonstrate Surya achieving an extraordinary Word Error Rate of 2.61% on Sinhala datasets, drastically

outperforming commercial alternatives like Google Cloud Vision API and open-source systems like Tesseract. Surya's superiority stems from its advanced layout analysis, precise reading order detection, and a deep learning architecture specifically optimized for complex character formations and table recognition, making it perfectly suited for parsing highly structured CVs.

Conversely, for Tamil document extraction, the benchmarking highlights Google Document AI as providing the highest accuracy, achieving a remarkably low CER of 0.78%. Should an open-source, locally hosted alternative be preferred for Tamil to avoid cloud latency and data privacy concerns, the `ocr_tamil` Python package offers highly robust text detection and recognition capabilities. It is specifically fine-tuned for Tamil typography, outperforming EasyOCR and Tesseract in both natural scene extraction (e.g., photographing a physical ID card) and standardized document parsing.

The optimal architectural implementation utilizes a dynamic routing function: upon receiving an image payload via WhatsApp, a lightweight computer vision classifier detects the primary script. Sinhala documents are instantly routed to a self-hosted Surya OCR container, while Tamil documents are passed to Document AI or the `ocr_tamil` pipeline. For standard, text-based PDF resumes, leveraging dedicated parsing tools like LlamaParse ensures that complex layouts, nested tables, and formatting structures are extracted into clean Markdown or JSON before being fed into the LLM, entirely bypassing the need for image-based OCR.

Dynamic Retrieval-Augmented Generation (RAG) for Real-Time Synchronization

A fundamental requirement of the recruitment chatbot is possessing continuous, up-to-the-minute knowledge of the business's operational state—specifically, newly created job projects, modified candidate requirements, and updated corporate policies. Relying on the static knowledge embedded within an LLM's weights is insufficient, necessitating the implementation of a highly dynamic Retrieval-Augmented Generation (RAG) architecture.

The Limitation of Static Knowledge and Naive RAG

Standard LLMs suffer from severe knowledge stagnation; their internal parametric memory is frozen at the precise moment their training concludes. They cannot

answer queries regarding a job role created yesterday. Naive RAG systems attempt to solve this by dumping static PDFs or job descriptions into a vector database and performing basic similarity searches. However, in a fast-paced recruitment agency, job statuses change hourly—requirements are updated, positions are filled, and salary bands are adjusted. If the vector database is not tightly synchronized with the operational database (the ATS), the LLM will hallucinate confidently, offering candidates roles that no longer exist or providing incorrect qualification prerequisites. A system that misleads a candidate destroys user trust immediately.

Architecting Real-Time, Dynamic RAG Infrastructure

To guarantee the recruitment agent operates with the exact awareness of a human recruiter, the architecture must bridge structured, real-time operational data with semantic vector search.

First, the system requires a real-time data ingestion pipeline. Instead of relying on manual batch uploads of job descriptions, the system must utilize streaming architectures or webhooks directly from the ATS. Whenever a job is added, modified, or closed in the CRM, an event payload is triggered. A dedicated background Celery task intercepts this payload, processes the text, and generates a dense vector embedding using a highly performant embedding model (such as Cohere Embed or OpenAI's `text-embedding-3-small`). The system then performs an incremental 'upsert' (update or insert) into the vector database, such as Pinecone, Milvus, or a PostgreSQL database equipped with pgvector. This incremental synchronization is vital; it prevents the computationally exhaustive and latency-inducing need to re-index the entire knowledge base for every minor update.

Secondly, the way data is chunked before embedding dictates retrieval accuracy. Job descriptions should never be split arbitrarily by character count, as this severs critical context. Semantic and hierarchical chunking strategies must be employed, breaking documents at natural topic boundaries. Crucially, rich metadata—such as the Job ID, Location, Required Experience, and Active Status—must be explicitly attached to every vector chunk. This ensures that the retrieval algorithm does not lose the context of exactly which job a specific requirement belongs to, and allows the system to pre-filter searches to only include "Active" roles before performing the vector similarity search.

Hybrid Search and Advanced Re-Ranking Mechanisms

Relying solely on dense vector search, which maps semantic similarity, often fails in the recruitment domain where exact keyword matches are critical. A candidate searching for "React developer" might miss out if the semantic search over-indexes on general "software engineering" roles. The retrieval pipeline must implement Hybrid Search—combining dense vector embeddings for deep contextual understanding with sparse lexical techniques, such as BM25, for precise exact-keyword matching.

To further refine the results, a two-stage retrieval process is highly recommended. In the first stage, the hybrid search rapidly casts a wide net, retrieving 20 to 30 potentially relevant job descriptions from the vector database. In the second stage, a dedicated Cross-Encoder Re-ranking model evaluates these candidate documents against the user's specific query. Cross-encoders are computationally heavier but vastly more accurate, scoring the exact relevance of each document pair and surfacing only the top three most highly correlated job descriptions to inject into the LLM's context window. This ensures the LLM is only fed the highest quality data, drastically reducing hallucination rates and generating hyper-relevant recommendations for the candidate.

To achieve true autonomy, the system can evolve toward Dynamic RAG (DRAGIN), an agentic approach where the LLM actively decides *when* to trigger a retrieval based on its own uncertainty regarding a user's question, and dynamically formulates the specific search query needed to find the missing information, rather than blindly retrieving data for every single user message.

Seamless Recruitment System Integration and Autonomous Orchestration

The ultimate commercial utility of this chatbot lies in its ability to transcend simple question-answering and act as a fully autonomous recruitment agent—screening candidates intelligently, extracting necessary data from conversational flows, and pushing perfectly structured payloads into the recruitment CRM seamlessly without human intervention.

Agentic Workflows and Conversational State Management

The chatbot architecture must be modeled as an Agentic workflow capable of multi-turn, goal-oriented data collection. When a message arrives, a high-speed, lightweight classification prompt determines the user's intent—for example, distinguishing between a candidate inquiring about general company culture (`general_faq`) and one expressing interest in applying for a specific role (`submit_application`).

When an application intent is triggered, the bot enters a persistent "slot-filling" state, utilizing backend session memory (stored in Redis) to track the conversation. The agent systematically and conversationally asks the candidate for required data points: name, email, preferred job role, years of experience, and a CV upload. The agent adapts its questions dynamically based on the candidate's previous answers; for example, if the candidate states they have five years of nursing experience, the agent immediately utilizes the RAG pipeline to query the database for senior healthcare roles, skipping irrelevant IT positions entirely.

Structured Data Extraction via JSON Schemas

To interface correctly with strict ATS software, the highly unstructured data generated by conversational natural language, code-mixed audio transcripts, and OCR-extracted CV text must be synthesized into rigid, predictable formats.

Large Language Models excel at this transformation when heavily constrained. By utilizing framework features like OpenAI's Structured Outputs or strictly engineered system prompts specifying JSON schemas, the LLM is instructed to parse the entire conversation history and the raw text of the uploaded CV, outputting nothing but a pure, validated JSON payload.

An example of the output schema generated by the LLM orchestration layer:

JSON

```
{
  "candidate_profile": {
    "first_name": "Nuwan",
    "last_name": "Jayawardena",
    "contact_number": "+94771234567",
    "email_address": "nuwan.j@example.com",
    "primary_languages":,
    "extracted_skills":,
    "years_experience": 3,
    "matched_job_requisition_id": "REQ-2045"
```

```
  },
  "missing_critical_information": ["highest_education_level"]
}
```

The application logic evaluates this JSON object before making any external API calls. If the `missing_critical_information` array is populated, the orchestration layer intercepts the process, preventing a database write, and prompts the WhatsApp user to provide the missing education details to complete their profile. This guarantees that the CRM only receives complete, high-quality candidate records.

CRM Synchronization and Multimedia Document Management

Once the JSON payload represents a complete and verified candidate profile, the backend system utilizes the CRM's native REST APIs to create a new record, mapping the JSON keys directly to the ATS database fields. This entirely automates the data entry process, eliminating hours of manual transcription for HR personnel.

Handling multimedia attachments securely and permanently is a vital final step. Media files shared via the WhatsApp Business API are hosted on temporary links that eventually expire. When a candidate uploads a document or image—such as a PDF CV, an image of an identification card, or a portfolio file—the FastAPI backend must immediately execute a secure HTTP GET request to download the raw file from Meta's servers. The system then parses the file via the established OCR or LlamaParse pipelines to extract the text for the LLM.

Crucially, the raw, original file is then programmatically uploaded to the recruitment agency's permanent cloud storage infrastructure (e.g., an AWS S3 bucket) and physically linked to the candidate's newly created profile within the CRM. This architecture ensures that the HR team has permanent, organized access to original documents, which are instantly downloadable and ready to preview within the CRM dashboard, fulfilling all compliance and verification requirements without ever leaving the primary software ecosystem. By automating these repetitive administrative burdens and executing them with high precision, the AI agent allows human recruiters to focus entirely on high-value candidate relationship building and final-stage interviewing.